Problem Tracker — Segment 5: The Interesting Technical Decision

5 of 8 · 6 min

SM-2 vs. the scheduling heuristic · pairs with `docs/scheduling-comparison.md`, `sm2.py`, `sm2_vs_heuristic_demo.py`

> **Opening line (say this before showing anything):** "This is my favorite story in the project, because it's not 'I used algorithm X' — it's 'I implemented the actual canonical algorithm standalone, benchmarked it against my own hand-rolled heuristic on real data, and kept the heuristic anyway, for reasons I can defend.'"

Before You Start

- Have a terminal ready in the project root with the venv active — you'll live-run `python3 sm2_vs_heuristic_demo.py` for real output, not a screenshot.
- Run it once beforehand so you're not surprised: numbers drift slightly day to day, but the *shape* of the finding (real divergence, plus a real limitation in canonical SM-2 itself) won't.
- Know the one-line contrast cold: **SM-2 answers "is this due?" per item; the heuristic answers "which top-N across everything?"** — that's the whole reason a straight swap doesn't work.

The 6-Minute Run of Show

0:00–0:30	Framing — why this is the best story
0:30–1:30	How SM-2 works
1:30–2:15	How the heuristic works
2:15–2:45	The lossy outcome→quality mapping
2:45–3:45	Live run — 59 real divergences
3:45–4:45	Two real examples + SM-2's ceiling bug
4:45–5:30	Why the heuristic was kept (4 reasons)
5:30–6:00	Where SM-2 wins + close

How SM-2 works (0:30–1:30)

- *"Each item carries three numbers: repetitions, interval in days, and an ease factor starting at 2.5. Every review is graded 0–5. Below 3, the schedule resets to day 1. At or above 3: 1st pass → 1 day, 2nd → 6 days, 3rd+ → previous interval times the ease factor."*
- *"The output is a due date per item — it answers 'is this due yet,' not 'what should I study today.' That distinction turns out to be the whole ballgame."*

How the heuristic works (1:30–2:15)

- *"score_row computes one continuous urgency number per problem, every dashboard load: age since last review times two, plus 10 if starred, plus 20 for a Failed or 10 for a Hint, minus a capped discount per past solve, plus 5 per past struggle. build_queue sorts by that and caps how many problems per category can appear — no ten Trees problems in a row."*
- *"SM-2 has no equivalent to either the starred boost or the category cap. Both got bolted on because the app needed them, not because a scheduling algorithm handed them out."*

The lossy outcome→quality mapping (2:15–2:45)

- *"The app logs one of four discrete outcomes — Solved, Hint, Failed, Skip. SM-2 wants a continuous 0–5 self-rating. I mapped Solved→5, Hint→3 — SM-2's own floor for 'still passing' — Failed→1, and Skip is excluded entirely: no attempt means it shouldn't advance or penalize the schedule, so the mapping function raises rather than guess."*
- *"That mapping is itself a real design decision, and it's lossy. It's also not what feeds the comparison below — review_history is only 22 rows, a few weeks old. The demo instead replays accepted_dates, LeetCode/NeetCode's own record of every accepted submission, which turned out to hold real multi-year history: every accepted_dates row is a successful submission by definition, so that replay is a flat quality-5 each time, not this mapping."*

Live run — 59 real divergences (2:45–3:45)

```
$ python3 sm2_vs_heuristic_demo.py
problem                      heuristic reps  EF  SM-2 next due  SM-2 days
Container With Most Water        129.0    7 3.20   2030-04-10       1306
Coin Change                      206.0   10 3.50   2183-04-18      57196
Subsets                          126.0   11 3.60   2591-04-16     206213
...
59 of 354 problems show heuristic/SM-2 divergence.
```

- *"149 of 354 problems turned out to have 2 or more distinct accepted dates — some going back to 2020. That's real multi-year repetition history, and it's exactly the 'mature dataset' this doc used to say didn't exist yet. Once I pointed the comparison at that instead of the app's own thin review log, 59 problems showed genuine divergence."*
- *"I didn't go looking for a divergence to cherry-pick — I found this by building a completely unrelated feature, accepted-date history for the 'Solved 0x' bug, and then it occurred to me the richer data was sitting right there for this comparison too."*

Two Real Examples + SM-2's Ceiling Bug (3:45–4:45)

Container With Most Water

7 distinct accepted dates on record (real LeetCode history).

Heuristic: capped solve-discount → still **score ≈ 129, flagged urgent**
SM-2: `replay([5]×7)` → EF 3.2 → **not due for 1,306 more days (~3.6 yrs)**

vs

Subsets

11 distinct accepted dates, 2020–2026 (the most of any tracked problem).

Heuristic: same capped discount → **score ≈ 126, flagged urgent**
SM-2: `replay([5]×11)` → EF 3.6 → **not due for 206,213 days — the year 2591**

"Both real, not constructed. The heuristic's discount caps at -10 regardless of solve count, so 7 and 11 solves look nearly identical to it. SM-2's interval compounds multiplicatively via EF, so that gap keeps growing forever — canonical SM-2 has no maximum interval. 206,213 days isn't a bug in my implementation, it's the 1987 spec doing exactly what it says with enough clean reps. Anki caps this in practice for exactly this reason — asked 'would you ship raw SM-2,' the honest answer is no, not without that same cap."

Why the heuristic was kept (4:45–5:30) — rapid-fire

- **Needs a ranked top-N, not a flag.** Given a capacity of 5–20, picking the most valuable items across everything due still needs a ranking step on top of SM-2 — which is exactly what score_row already does.
- **Category balance has no SM-2 equivalent.** The per-category cap would have to be bolted on regardless of which scheduler ran underneath — switching wouldn't remove that custom logic, only add SM-2 alongside it.
- **Not cold-start anymore, but no ceiling either.** This used to be 'the dataset's too young to tell' — it isn't now. The reason shifted: raw SM-2 has no max interval, and on real data it schedules things for the 26th century. Adopting SM-2 well means adopting an interval cap too — that's extra logic on top of SM-2, not a reason SM-2 alone is simpler than the heuristic.
- **State cost.** SM-2 needs mutable per-item state persisted and updated every review. The heuristic derives its score fresh, statelessly, from the review log already kept for the notes field.

Where SM-2 wins + close (5:30–6:00)

- *"If the goal shifted from 'structured breadth across topics' to 'retain each problem's pattern in long-term memory with minimal repetition,' SM-2's EF-based growth is a validated model of the forgetting curve — the heuristic's weights are hand-tuned constants with no research behind them."*
- *"That's a deliberate trade, and sm2.py exists so it's a real, demonstrable comparison — not just an assertion. Happy to go deeper on either algorithm."* — invite questions.

If they ask "then why build SM-2 at all if you weren't going to use it?" — "I considered X and rejected it" is only a real claim if you can show the comparison, not just assert it. A standalone, tested SM-2 implementation plus a script replaying real history through both is the difference between "I thought about SM-2" and "I can show you exactly where it would've changed today's queue."